Underworld and Singularity

Julian Giordani[1]

[1]University of Sydney

DOI https://doi.org/10.6084/m9.figshare.33193545

Abstract

TL; DR: Underworld is now Singularity-enabled, making it easier and somewhat quicker to use compared to traditional HPC installs.

Keywords Underworld Code, Tricks of the Trade

PUBLISHED Apr 09, 2025
ARTICLE UWTN 2025-001
VERSION 1.0.0
LICENCE CC-BY-4.0
ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/underworld-and-singularity/

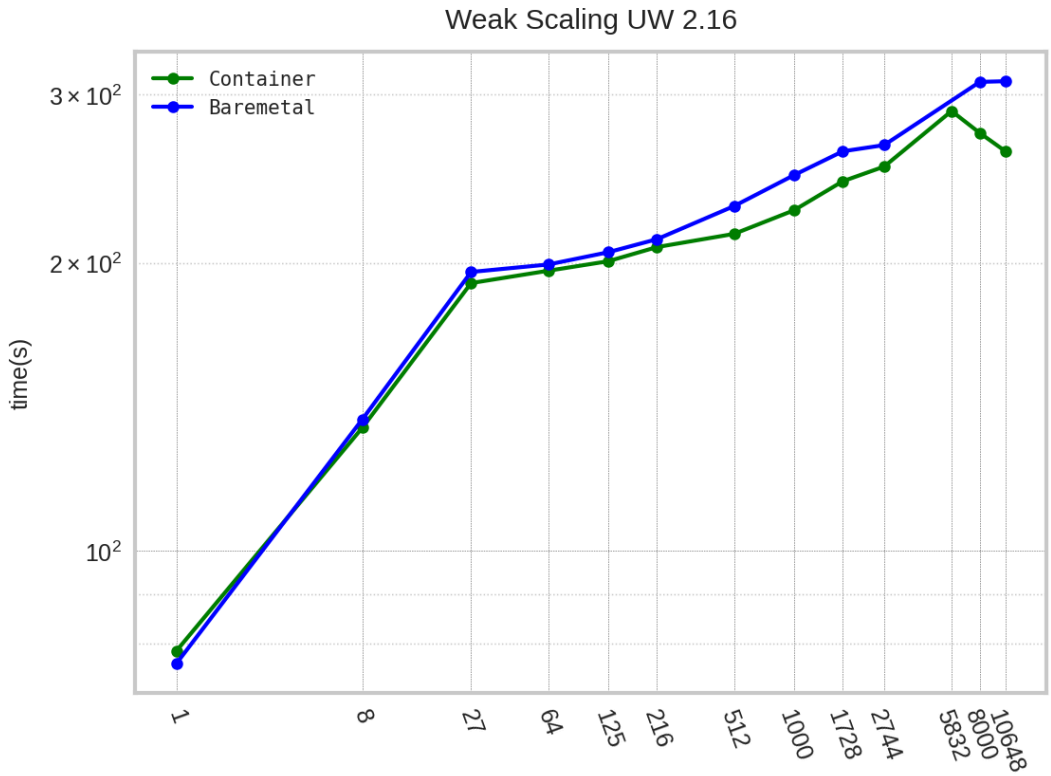

Figure 1: Fig 1. Total Runtime of timed_model.py with Underworld 2.16. Container (Singularity) is quicker than default (traditional) Baremetal.

1. Benefits of Containers

1.a. ***Pros:***

- Simpler to install Underworld and run with Singularity containers.
- No need to maintain “bare metal” (hard drive) builds.
- More efficient Python execution.
- Increased reproducibility and portability.

1.b. ***Cons:***

- Non-performant compilation for HPC hardware configurations.
- Static software, with modifications only possible at execution time or via container rebuild.

In the last decade, container technology (e.g., Docker) has revolutionised computing workflows. Computational sciences have greatly benefited from containers in terms of usability, distribution, and reproducibility. The Underworld team has been at the forefront of this trend, maintaining containers for all recent releases.

Underworld2 is now running HPC-enabled containers via Singularity on Australia’s two eminent research supercomputers: Gadi (NCI) and Setonix (Pawsey). The benefits of containerisation outweigh the negatives, and in this post we highlight this and present the new setup for running Underworld2 on Gadi and Setonix.

Fig 1.

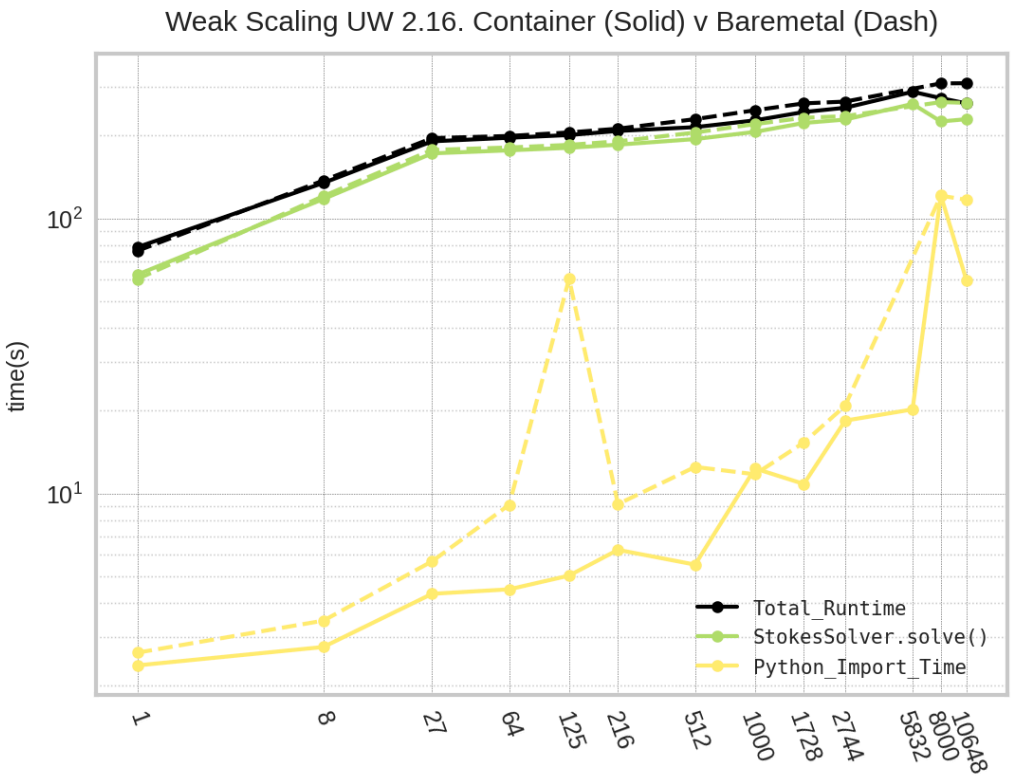

Figure 2: Weak scaling measures parallel efficiency. Increasing cpu number on the horizontal axis and runtime on the vertical axis.

Fig 1. is a weak scaling plot, where a model with constant work per CPU is timed on increased numbers of CPUS (1 to 10,648). The horizontal axis represents CPU counts and the vertical axis represents total run time. Weak scaling measures parallel communication efficiency of computation, a horizontal plot indicate an algorithm that scales perfectly with more CPU (with computation per CPU kept constant on communication overheads should cause it to increase in time).
The model used in this case runs all of Underworld's significant algorithms, the most costly of which is solving the stokes equation (the green graph).
From the plots we find the execution of singularity (solid lines) is slightly more efficient than the tradition / bare metal installs of Underworld 2.16. The significant difference are seen in the Python_Import_Time, yellow line, which, to be clear, is not a part of the Total_Runtime.
This is an encouraging result. Overall parallel efficiency with singularity is no less performant than the bare metal install and the Underworld python package import time, a startup costs, sees significant improvement.

This results, along with the simplified workflow of using singularity images makes it a clear best practice for running Underworld models on HPC platforms.
Below we showcase how to run Underworld2 with Singularity on Gadi or Setonix.

---

2. Key Ingredients for a Singularity Build:

(Developer notes)

For container creation, Dockerfile considerations.

1. ABI-Compatible MPI Implementation: Your container must be built with an ABI-compatible MPI implementation for the HPC machine. For Gadi, use OpenMPI; for Setonix, use MPICH.
2. Bind Mounts and Path Variables: Proper bind mounts and path variables must be set for HPC machine MPI utilization. Different HPC systems automate these setups individually, so care must be taken. Hence the different commands in the runner.sh scripts above.

For singularity execution.
Bind Mounts:
The –bind argument when running *singularity* forces directories on the host machine to map into the running container filesystem. For Underworld, we map the HPC's MPI implementation directories into our running container on Gadi to access the high performance network of the HPC.